Left-Right: Wait-free Reading While Writing

2018-05-01 06:15:25

Original: `https://nicknash.me/2018/05/01/left-right-wait-free-reading-while-writing/`

1 Recap

In some previous posts (here and here) I've described Pedro Ramalhete and Andreia Craveiro's Left-Right, a concurrency construction that enables concurrent reading and writing of a data structure, while keeping reads wait-free. My last post described a variation of Left-Right where a writer may be forced to wait indefinitely – i.e., be starved. In this post, I'll describe how writer starvation can be avoided.

2 Left-Right Features Refresher

As a quick refresher (once again), the key features of Left-Right are:

1. It's generic, works with any data structure, and doesn't affect the time complexity of any operations.
2. Reads and writes can happen concurrently.
3. Reading is always wait free.
4. No allocation is required, so no garbage is generated, and no garbage collector is required.
5. A writer cannot be starved by readers.

In my last post, the implementation of Left-Right I presented had all these properties except the last. Let's see how write starvation can be avoided.

3 Solving Write Starvation

Let's remind ourselves how writing works on the variation of Left-Right I last described:

```
void Write()
{
    lock(_writersMutex)
    {
        var readIndex = _readIndex;
        var nextReadIndex = Toggle(readIndex);
        write(instances[nextReadIndex]);
        _readIndex = nextReadIndex;
```

```
        // This wait is the source of starvation:
        while(_readIndicator.IsOccupied) ;
        write(instances[readIndex]);
    }
}
```

Recall that readers mark `_readIndicator` to communicate their reads to the writer. It is the wait on `_readIndicator.IsOccupied` that opens up the possibility of writer starvation: new readers could continuously arrive, indefinitely delaying an in-progress write.

Clearly, for the writer to avoid starvation it cannot wait on a `_readIndicator` that new readers can mark. This need echos the original idea of Left-Right: keep two copies of the data structure so that writes can proceed during reads. So the key to avoiding starvation is to have *two* read-indicators also.

4 Reading

Reading is similar to with a single read-indicator, there is just one additional step, readers read a variable `_versionIndex` that determines which read-indicator they will mark:

```
// An array of the two instances of the data
// structure we are coordinating access to.
List[] _instances;
// An index (either 0 or 1) that tells readers
// which instance they can safely read.
int _readIndex;
// An index (either 0 or 1) that tells readers
// which read-indicator they should mark.
int _versionIndex;
// An array of two 'read indicators' so a
// reader can inform writers that it has a
// read in progress
IReadIndicator[] _readIndicators;
int Read()
{
    var versionIndex = _versionIndex;
    var readIndicator = _readIndicator[versionIndex];
    readIndicator.Arrive();
    var idx = _readIndex;
    var result = read(instances[idx]);
    return result;
    readIndicator.Depart();
}
```

This is so similar to the "No Version" read mechanism of the previous post that there isn't much to say about it, instead, let's look at writing, where the real differences lie.

5 Writing

Writing is again similar to in the simpler "No Version" Left-Right, but includes *two* waits. Roughly the way writing works is: Write to the instance we're sure no readers are reading, change `_readIndex`

so readers read the version we just wrote to, wait for any outstanding reads on that `_readIndex` we want to write next, change `_versionIndex` so readers mark the other read-indicator, wait for the outstanding reads to clear, and do the final write. That explanation is a bit of a mess, hopefully the code makes things a little clearer:

```
void Write()
{
    lock(_writersMutex)
    {
        var readIndex = _readIndex;
        var nextReadIndex = Toggle(readIndex);
        write(instances[nextReadIndex]);
        _readIndex = nextReadIndex;
        var versionIndex = _versionIndex;
        var nextVersionIndex = Toggle(versionIndex);
        while(_readIndicator[nextVersionIndex].IsOccupied) ;
        _versionIndex = nextVersionIndex;
        while(_readIndicator[versionIndex].IsOccupied) ;
        write(instances[readIndex]);
    }
}
```

Hopefully writing makes some intuitive sense, getting a confident feel for *exactly* why it enforces both mutual exclusion between readers and writers, as well as ensures writing is starvation free is a little more effort. We'll try and do that in the next section.

6 Why This is Correct

6.1 Mutual Exclusion

The argument for correctness here is similar the the one for the version of Left-Right presented in the previous post. The key difference is that there is an additional piece of state to consider: `_versionIndex`. To keep the argument clear, let's start with some definitions.

Let's define the *end-state* of a reader as the values of `_versionIndex` and `_readIndex` it has respectively witnessed at the time it calls `Depart`. So there are four possible reader end-states: `(0, 0)`, `(0, 1)`, `(1, 0)` and `(1, 1)`. After a reader calls `Depart` it is said to *complete* and is not defined to be in any state.

Let's also divide readers into two categories: *pre-arrival* readers and *post-arrival* readers. Pre-arrival readers have not called `Arrive` on a read-indicator yet. While post-arrival readers have called `Arrive` on a read-indicator.

Let's introduce a small bit of notation for writes. By a writer "performing" a `Wait(V, R)` we mean the writer waited until `_readIndicator[V].IsOccupied` returned `false`, while `_readIndex` was equal to `R`.

Now we can note the *elimination property* of `Wait(V, R)`: After a writer performs `Wait(V, R)` then there can be no reader with end-state `(V, Toggle(R))`. This is because any pre-arrival readers will enter state `(V, R)`, and any post-arrival readers will complete.

Now consider the two waits performed during a `Write`. Let's refer to the values of `_versionIndex` and `_readIndex` at the beginning of the write as `V0` and `R0` respectively:

1. The first wait is of the form `Wait(Toggle(V0), Toggle(R0))`, which by the elimination property means there can be no reader with end-state `(Toggle(V0), Toggle(Toggle(R0)) = (Toggle(V0), R0)`.
2. The second wait is of the form `Wait(V0, Toggle(R0))`, which by the elimination property means there can be no reader with end-state `(V0, Toggle(Toggle(R0)) = (V0, R0)`

Now we can spell out how these waits enforce mutual exclusion. A write is mutually exclusive with a read if it is writing to instance `_readIndex` and there are no readers with end-states `(0, _readIndex)` or `(1, _readIndex)`. But these are exactly the end-states eliminated by the two waits performed during a `Write` before it performs its second mutation of the data structure (i.e., `write(instances[readIndex])`). Thus the second mutation is mutually exclusive with readers.

Given that this second mutation is mutually exclusive with readers, then clearly a subsequent call to `Write` can safely mutate this instance first, without waiting. This just leaves the very first mutation performed by the very first call to `Write`, which must be mutually exclusive with readers since it targets the toggle of the initial value of `_readIndex`.

The above stops short of formalising things into a proof (although it wouldn't be too much more effort), but it's enough to be fairly confident that Left-Right enforces mutual exclusion.

The whole point of this more complicated version of Left-Right compared to the one in my last post is that writers cannot be starved by readers, so let's look at that next.

6.2 Starvation Freedom

What is meant be starvation freedom of the writer by readers is that in any wait performed by a `Write` operation, that wait must end when *only* the readers that were in-progress at the moment it began waiting complete.

It's pretty clear both of the waits performed in `Write` satisfy this, both are of the form `Wait(Toggle(V), .)`, where `V` is the current value of `_versionIndex`. That is, while new readers are marking `_readIndicator[V]`, the wait depends only on those that have marked `_readIndicator[Toggle(V)]` before the wait began, and so the wait will end when these existing readers complete.

7 Next Time

There is still a large gap in the description of Left-Right given so far: it omits all details of memory fencing. The descriptions above all rely on viewing the code as sequentially consistent. An actual implementation is likely to diverge from this. I'll try and close this gap in a subsequent post, using RelaSharp to help. On this last point, generally speaking in my experience, a good working assumption for code implementing a mutex (or lock free or wait free code in general) is that any implementation that hasn't been model-checked in some way – even where the underlying construction has been proved correct – is buggy. The source code for this Left-Right implementation (including a simple RelaSharp test) is here, there's also a NuGet package available: SharpLeftRight